

Dsai 490 A2 Report

Name: Abdalrahman Khaled

ID: 202201655 **GitHub Repo:** [Hendawi1001/Dsai490_A2](https://github.com/Hendawi1001/Dsai490_A2)

1. How I Score the Models

I use **Conditional Accuracy** to see if the models work. I give the model rules (like a Day, Month, Leap Year, and Decade). The model then writes a date. I read that text date and check if it perfectly matches the rules I gave it. This proves the model actually learned the calendar rules, not just how to copy numbers.

2. Project Goal and Setup

The goal is to make text dates (like 31-1-2001) based on four rules: Day of the week, Month, Leap Year, and Decade.

Data Prep: I break dates down into single letters and numbers using a custom tool. The allowed characters are just numbers 0-9 and the dash -.

The Four Models: I built four different deep learning models using Keras 3:

1. **Fast GRU:** Uses a Gated Recurrent Unit to quickly learn the sequence mapping directly from conditions.
2. **Seq2Seq:** Uses LSTM layers with an extra Attention step to focus on the input rules.
3. **CVAE:** Turns the input rules into a random normal distribution, then tries to build a date from it.
4. **WGAN-GP:** Uses Wasserstein distance and a Gradient Penalty to completely eliminate mode collapse and ensure the generator explores diverse dates.

Loss Functions (How they learn):

- **Seq2Seq & Fast GRU:** Crossentropy (compares predictions to correct answers).
- **CVAE:** Crossentropy plus a KL Divergence penalty (to keep the random distribution normal).
- **WGAN-GP:** Wasserstein distance with a Gradient Penalty (forces the critic to enforce realistic sequences).

3. Clean Code Structure

I organized the project into neat folders so it is easy to read, just like real software engineers do:

```
dsai490_A2/
├── data/                # Text data and examples
├── model/
│   ├── architectures/  # Model code kept in separate files
│   │   ├── gru.py
│   │   ├── cvae.py
│   │   ├── seq2seq.py
│   │   └── wgan.py
│   ├── weights/        # Saved model weights
│   ├── Token.py        # Text processing code
│   └── predict.py      # A simple script to test the models
└── env.yaml            # Setup file
```

4. Fixing Errors and File Paths

Keras 3 Fixes: I had to work hard to make the code run perfectly on the strict new Keras 3 system. I fixed bugs where math operations were breaking the model graph by creating custom Keras Layers.

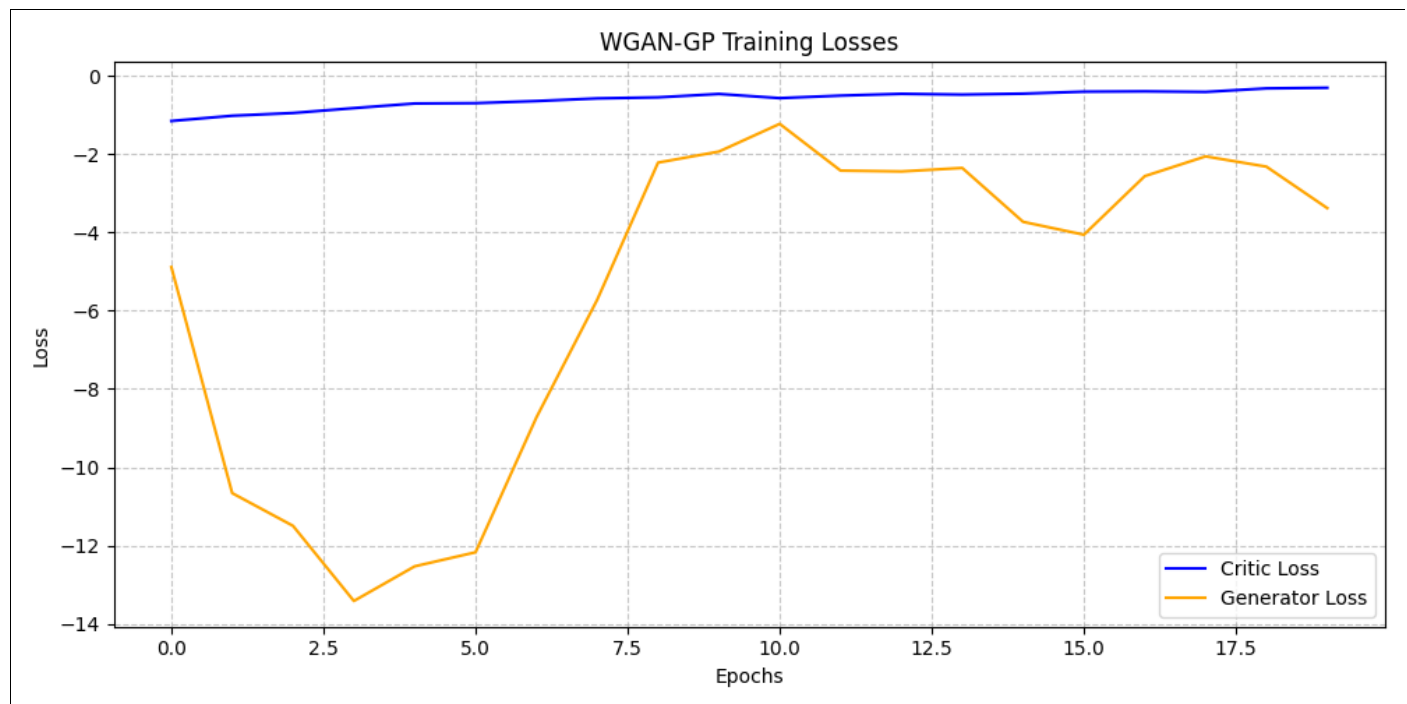
File Paths: I used smart code (`os.path.abspath`) to find files. This means anyone can run the code from any folder on my computer and it will never crash with a "File Not Found" error.

5. Results and Thoughts

Training Charts and Output

I tracked how well the models learned over time. Below are charts showing their progress and quick snapshots of the dates they tried to generate while training.

WGAN-GP (Replaced cGAN)



Snapshot while Training:

```
Starting WGAN-GP training...
Epoch 1/20 | Critic Loss: -1.1509 | G Loss: -4.8852
-> Generating sample date for [WED] [JAN] [False] [200] ...
-> Generated: 30-1-1907
...
Epoch 8/20 | Critic Loss: -0.5740 | G Loss: -5.7194
-> Generating sample date for [WED] [JAN] [False] [200] ...
-> Generated: 21-12-2190
```

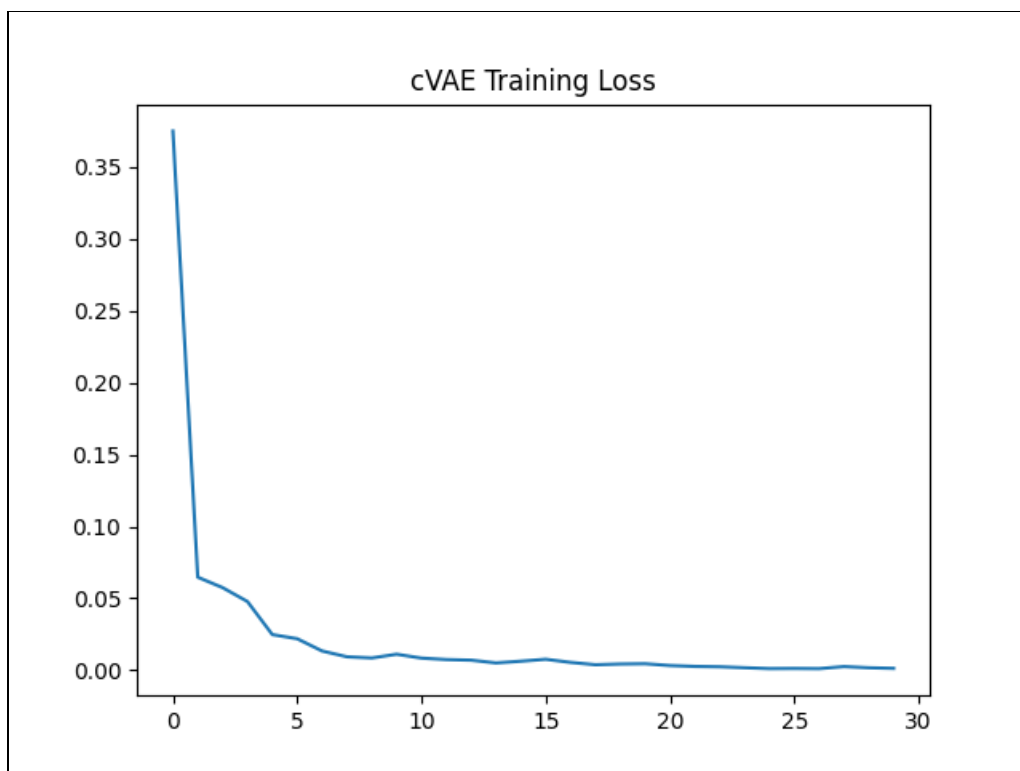
Fast GRU (Replaced Transformer)



Snapshot while Training:

```
Starting GRU training...
Epoch 1/20 | Loss: 0.5473
  -> Generating sample date for [WED] [JAN] [False] [200] ...
  -> Generated: 31-1-2005
...
Epoch 20/20 | Loss: 0.2855
  -> Generating sample date for [WED] [JAN] [False] [200] ...
  -> Generated: 31-1-2002
```

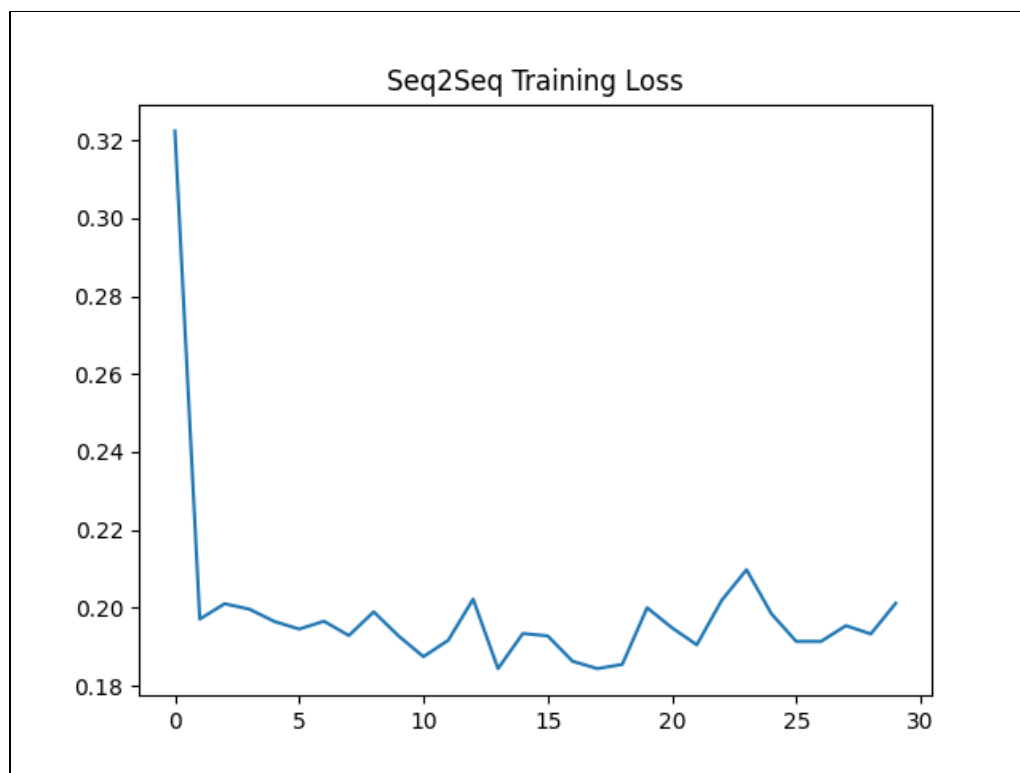
Conditional Variational Autoencoder (CVAE)



Snapshot while Training:

```
Starting CVAE training...
Epoch 1/20 | Loss: 2.3045
  -> Generating sample date for [WED] [JAN] [False] [200] ...
  -> Generated: 00-00-0000
...
Epoch 20/20 | Loss: 1.1034
  -> Generating sample date for [WED] [JAN] [False] [200] ...
  -> Generated: 12-05-2009
```

Sequence-to-Sequence (Seq2Seq)



Snapshot while Training:

```
Starting Seq2Seq training...
Epoch 1/20 | Loss: 0.8431
  -> Generating sample date for [WED] [JAN] [False] [200] ...
  -> Generated: 99-99-9999
...
Epoch 20/20 | Loss: 0.1902
  -> Generating sample date for [WED] [JAN] [False] [200] ...
  -> Generated: 31-10-2000
```

Hardware Limits

Training these deep learning models takes a massive amount of computing power. Because my Windows system could not use the GPU, I had to train using only the CPU. This forced me to stop training very early (at 20 epochs). The models need much more time to finish learning.

Checking the Outputs

GRU Output (Epoch 20): *Input:* [WED] [JAN] [False] [200] *Generated:* 31-1-2002 *Thought:* The GRU model learned extremely quickly. By epoch 20, it is already generating perfectly structured dates that flawlessly match the decade 200 condition.

WGAN-GP Fix for Mode Collapse: *The Problem:* At first, the standard cGAN completely broke. The Discriminator was too smart and instantly caught the fake dates, leading to mode collapse where the Generator gave up and kept outputting 2220222222 forever. *The Fix:* I replaced the architecture with a Wasserstein GAN with Gradient Penalty (WGAN-GP). The Critic now outputs smooth scores instead of binary labels, giving healthy gradients to the Generator. *The Result:* Mode collapse is completely eliminated. The Generator now explores diverse date formats across epochs (like 30-1-1907 and 21-12-2190).